University of Mons
Sciences Faculty
Computer Science Department

Probabilistic Verification of Network Configuration

Explanatory Report

Author :
Maxime Bartha

Supervisor :
Bruno Quoitin

Academic Year 2025-2026

Contents

1 Intro **2**

2 Probabilistic Verification of Network Configuration **3**

2.1 Incorporating BGP 7

2.2 Complex Link Failure Probability model 12

2.3 Results 12

1 Intro

Table 1: Properties supported by NetDice

	Property	Meaning
Single-Flow	$Reach_{u \to d}$	Reachability from source u to destination d
	$Waypoint_{u \to d}(w)$	Traffic from u to d traverses w
	$Egress_{u \to d}(e)$	Traffic from u to d leaves the network at e
	$PathLength_{u \to d}(l)$	Traffic from u to d traverses l links
Mutli-Flow	$Balanced_F(\mathcal{L}, \Delta)$	Under flows F with given volumes, the load on the links in $\mathcal{L}$ differs by at most Δ
	$Isolation_F$	Set of flows in F doesn't share any links
	$Congestion_F(l, t)$	Together, all flows in F with their volume don't exceed threshold t for link l

2 Probabilistic Verification of Network Configuration

In this section, we explore the reasoning and implementation behind NetDice [3], a probabilistic verification tool for networks configuration.

It is able to verify properties on flows with respect to certain nodes or links in a network. A flow in the context of internet routing is generally defined by a source/destination IP address, port numbers and a transport protocol. We use a simplification considering the source and destination IP addresses only. NetDice implements the verification of properties defined by a single or multiple flows. Examples of properties and their meaning are represented in Table 1.

In NetDice, equal-cost multiple path routing (ECMP) is considered, hence a flow in a network can take multiples paths if they share the same cost. To represent such behavior, a forwarding graph, composed of V_{fwd} a set of vertices (routers) and E_{fwd} a set of directed edges (links), is used. A forwarding graph can be altered by link failures represented by a failure scenario which is a set of unusable links in the network configuration.

To compute the probability of a certain failure scenario, two mathematical tools are used :

1. A vector L of random variables $s.t.$ $L_e \in \{0, 1\}$ for each link e to represent if the link is up ($L_e = 1$) or down ($L_e = 0$)

2. A probability mass function of link failure $P(L)$

Such that, for a failure scenario represented by a vector l, its probability is defined by $P(L = l)$. This probability mass function is joint meaning it can represent complex relation between link failures, such as node failure (all incident links) or shared risk link group (SLRG) which are discussed in section 2.2.

Given a network configuration, a property ϕ and a link failure distribution L, the goal of NetDice is to compute the probability of the property being satisfied.

A naive approach could be to compute all possible failure scenarios and sum the probabilities of the scenarios for which the property holds. But the key insight of NetDice is to prune the exploration of failure scenarios where the state of certain links is guaranteed to have no impact on the state of ϕ. To this end, NetDice introduces the concept of cold edge.

Definition 1 *Given a network configuration with a certain failure scenario, a property ϕ and a flow (u, d), a subset of edges $\mathcal{C}$ is cold iff any combination of failures in $\mathcal{C}$ is guaranteed not to change the forwarding graph for (u, d). An edge is called cold if it belongs to $\mathcal{C}$ and hot otherwise.*

In practice, hot edges are used to prune the exploration of the possible failure scenarios. Intuitively, as they impact the forwarding graph, we only explore failure scenarios where hot edges fail to avoid redundant scenarios due to the unchanged forwarding graphs. As, if the forwarding graph doesn't change, a cold link could be up or down, the property won't change. Hence, The scenarios exploring cold links are disregarded.

An algorithm to compute hot edges considering the shortest paths and static routes is presented in Alg.1. It takes a flow (u, d), the set of static routes $Static_d \subseteq V \times V$ towards a destination d, a forwarding graph's edges E_{fwd} considering the static routes and a failure scenario l as parameters.

As the property holds depending on forwarding graphs, it is sufficient to consider as hot any edge that may make the forwarding graph vary. Hence, all edges along the shortest path and the traversed static routes are hot as a failure in any of them would change if ϕ holds due to unreachability or changes in the forwarding graph.

To compute the edges along the shortest path we define two algorithms :

Definition 2 $\mathrm{SP}_l(s, t)$ *returns the set of all edges along the shortest paths*[1] *from s to t, under link states defined by l.*

[1]Considering ECMP, there might be multiple equal-cost shortest paths

Definition 3 $\text{AllSP}_l(\mathcal{S}, \mathcal{T})$ *returns the union of* $\text{SP}_l(s, t)$ *for every* $s \in \mathcal{S}$ *and* $t \in \mathcal{T}$.

The algorithm starts by considering hot every edge along the shortest paths, considering the failure scenario l, from the source u to the destination d(Lin.2). As the shortest path is considered by u, any failure in this shortest path will impact the decision of the source node hence the resulting forwarding graph. It holds even if the initially shortest path does not correspond to the final forwarding graph, as static routes can deviate the routing behavior. Then edges along the shortest path from every end of a traversed static route are added as hot (Lin.4), as those static routes deviate the forwarding graph and the end-node considers the shortest path just as u initially. Finally, every edge on traversed static route is added as hot (Lin.5), as a failure would clearly change the forwarding behavior.

Algorithm 1 Hot edges for static routes and shortest path

1: **procedure** HOTSPSTATIC(u, d, $Static_d$, E_{fwd}, l)
2: $\quad \mathcal{H} \leftarrow \text{SP}_l(u, d)$ ▷ initial shortest path
3: $\quad \mathcal{S} \leftarrow \{y \mid (x, y) \in Static_d \cap E_{fwd}\}$ ▷ end nodes of static route
4: $\quad \mathcal{H} \leftarrow \mathcal{H} \cup \text{AllSP}_l(\mathcal{S}, \{d\})$
5: $\quad$ **return** $\mathcal{H} \cup (Static_d \cap E_{fwd})$

As an example, Figure 1 represents the application of Alg 1 giving the hot and cold edges of a certain network configuration considering no failure and a flow (D, C), the forwarding edges $E_{fwd} = \{(D, E), (E, B), (B, C)\}$ and the static routes towards C $Static_C = \{(E, B)\}$. Note that the (E, C) edge is marked as hot even though it is not in the forwarding graph because it lies on the shortest path of D. If it fails, the decision of D would be to use $D \rightarrow A \rightarrow B \rightarrow C$ as its shortest path, changing the forwarding graph.

Computing P(ϕ), intuitively, is done by only exploring possible failures of hot edges. A simple probability function mapping each edge to a certain failure probability is considered here, more complex probability failure model are discussed in Section 2.2. Moreover, a full algorithm incorporating BGP is presented in Section 2.1. To explore the failure space, NetDice considers 3 possible states for an edge during the exploration process :

- down : the edge cannot be traversed in current and future scenarios
- up : the edge can be traversed in current and future scenarios
- undefined : the edge is currently traversable but might not be in the future

Application of Alg 1

2: $\mathcal{H} = \{(D, E), (E, C)\}$

3: $\mathcal{S} = \{B\}$

4: $\mathcal{H} = \mathcal{H} \cup \{(B, C)\}$

5: returns $\mathcal{H} = \mathcal{H} \cup \{(E, B)\}$

5: returns $\mathcal{H} = \{(D, E), (E, C), (E, B), (B, C)\}$

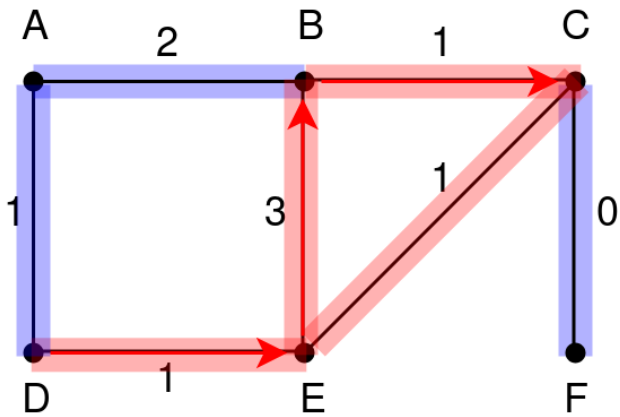


Figure 1: This example is from NetDice's article [3] it represents cold edges in a network configuration with no failures (i.e. all links up) and a flow (D, C). (E, B) is a static route. Blue and red highlight show cold and hot edges respectively. The forwarding graph is represented by the red arrows.

Those states are only considered during the exploration process. In terms of forwarding graph computation, within a certain scenario, down edges are not considered usable whereas up and undefined are considered usable. It is assumed that a *check* function returning, given a property and forwarding graphs, if a property holds in every forwarding graph is implemented. In practice, it is implemented using variants of DFS depending on the property to check.

The exploration procedure can be described as follows :

0. initially, all edges are undefined (all traversable)
1. compute the forwarding graphs with respect to the failure scenario
2. compute the hot edges with the forwarding graphs
3. let $\mathcal{H}^?$ be the set of undefined hot edges

4. $\forall h \in \mathcal{H}^?$ make a recursive call (going back to $pt1$) with a failure scenario considering h as down. Once the recursive call is finished (the scenario has been fully explored), set h as up in all future scenarios.

5. return the sum of the recursive calls and the probability of the actual scenario if ϕ holds (*check(ϕ)* is true)

Recall that multiple forwarding graphs are possibly computed as equal-cost multi-path is considered.

An example of execution on the network configuration from Figure 1 with property ϕ = "Traffic from D to C traverses E" is shown at Figure 2. The probability $\mathbf{P}(\phi)$ is computed with respect to the returning order of the recursive calls.

As required, initially all edges are undefined, the hot edges set is computed and recursive calls are made for every hot edge. At the first recursive call (D $\neq$ E), we observe that its first recursive call on (A $\neq$ D) results in a scenario where C isn't reachable from D. This means that every edge is cold, as any failure wouldn't change the forwarding graph which is empty. This leads to a dead end in the exploration as no more hot edge can be marked as down and the returned probability is 0 as the property doesn't hold. A similar situation happens in the very last recursive call (B $\neq$ C), except it's due to a loop in the forwarding graph because E uses its static route and B tries to reach C by E as (B - C) is down.

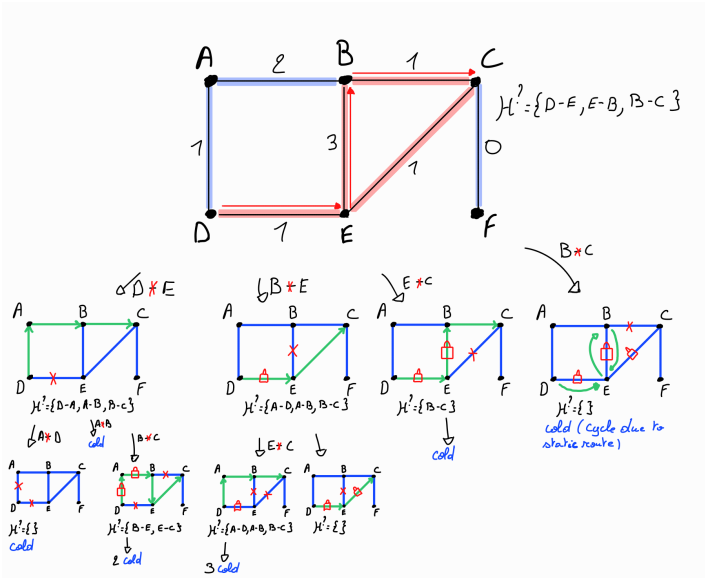
In the example, only 16 failure scenarios are explored. In contrast to the $2^7 = 128$ scenarios a Brute Force algorithm would have had to explore.

2.1 Incorporating BGP

To improve our last model with BGP, we now consider a flow to be defined by (u, d) with u an internal node and d a BGP destination. We consider the next-hop for internal routers to be the egress router towards the BGP destination. NetDice implements a simulation of BGP assuming that all external BGP announcement arrive simultaneously and that the propagation of such announcements is done step-by-step in the network [2]. The forwarding graph is computed after the end of the BGP simulation, as the selected next-hop is the next destination of a node towards d.

For the simulation, the BGP decision process described in Table 2 is used. The implementation of the BGP simulation of NetDice uses route reflectors [1].

The definition of cold and hot edges stands when considering BGP. The key difference BGP makes is the possible forwarding graphs changes occurring



P(ϕ) with independent probability of link failure p=0.1, $\tilde{p} = 1 - p$

Scenario	**Probability returned**	**Explanation**
(1,2,3,4):	0	only cold scenarios
(5):	$\tilde{p}^4 * p^2$	ϕ holds with 4 links up and 2 down
(6):	$\tilde{p}^4 * p^2$	ϕ doesn't hold, returns (5) result
(7,8,9):	0	only cold scenarios
(10):	0	ϕ doesn't hold
(11):	$\tilde{p}^2 * p$	ϕ holds with 2 links up and 1 down
(12):	$2(\tilde{p}^2 * p)$	same case as (11) + return of (11)
(13):	0	cold scenario
(14):	$\tilde{p}^3 * p$	ϕ holds with 3 links up and 1 down
(15):	0	cold scenario
(16):	(6)+(12)+(14)+$\tilde{p}^4$	sum of recursive calls, ϕ holds

P(ϕ) ≈ 0.897561

Figure 2: Exploration of failure space and computation of the probability with ϕ = "Traffic from D to C traverses E". Crosses represent down links, padlocks represent up links and green arrow represent the forwarding graph. Absence of symbol represent undefined links. "Cold" in blue represent scenarios containing only cold edges.

Table 2: BGP decision process. Announcement are compared in numerical order starting at 1. BGP path attributes are in italic.

Order	**Rule**
1	Prefer lowest *local pref*
2	Prefer Shortest *AS path*
3	Prefer lowest *origin*
4	Prefer lowest *MED* (only compared with same announcer AS)
5	Prefer external (eBGP) over internal (iBGP) announcements
6	Prefer lowest IGP cost to BGP "next-hop"
7	Prefer lowest *BGP peer ID* (tiebreaker)

if a selected next-hop changes. There are 3 reasons why a link failure could change the selected next-hop of a node :

1. If it causes a disconnection from a BGP peer (will induce the withdrawal of all routes learned by this peer)

2. If the IGP cost towards its current next-hop increases (could change the decision process of BGP)

3. The cascading effect of the two above points : if a node selects another next-hop, it will announce it to its neighbors.

Considering those new possibilities for a forwarding graph to change, the algorithm to compute the hot edges have to assess two properties. (i) keeping the connectivity between BGP peers and (ii) the IGP costs towards a selected next-hop stay unchanged. So, in addition to the previous requirements, an edge is now hot if its failure would falsify one of those two properties.

The set of network border routers, route reflectors and external BGP nodes are represented respectively by B_R, R_R and E_{xt}.
The algorithm to compute the set of hot edges considering a certain failure scenario l, a flow (u, d) and the forwarding graph E_{fwd} is presented in Alg.2.

To reduce computation costs, only the set of nodes X reachable with IGP by u is considered (Lin.2).

The algorithm uses the ALLSP from Definition 2 and TOP3 which returns, for a certain destination, the border routers announcing routes of with the same highest preference in the three first rules in the BGP decision process at Table 2 considering the subset of nodes X. It is used to filter out of

the computation the border routers whose announcement won't be selected as the three first rules are global. Its reasoning is similar to Alg.1 with the addition of the consideration for the connectivity and unchanged IGP cost properties. To make sure that all the route reflectors receive the announcements of the border routers, the edges along all the shortest paths between every route reflector and border router is marked hot (Lin.3-5). As in the first algorithm, we need to mark as hot the edges along the shortest paths, towards their selected next-hop, of u and the end-nodes of traversed static routes (Lin.6-7). In addition, the forwarding graph E_{fwd} can deviate if two successive nodes along it differ in selected next-hop. As the shortest path of the first might not match the shortest path of the second as their selected next-hop differs. To take this deviation into account, the edges along the shortest path of the second node towards its selected next-hop are marked as hot. Remark that, contrary to static traversed routes, the edge of the two successive nodes with different selected next-hops isn't manually marked hot, as it lies on the shortest path of the first node. Finally, the edge case where no route reflectors are reachable by u is handled (Lin.12-13). In this case, the connectivity from u towards every border router is required. So every shortest path from u towards them is considered hot. The first property (i) is ensured by considering shortest paths. The second one is ensured because the lines 5 to 10 makes sure of the connectivity in the case of at least one route reflector in X. Where the lines 11 and 12 take care of the case where no route reflector lies in X.

An example of the algorithm can be found in Figure 3. The route reflectors are V and W with respectively {E, D, C, B, W} and {F, B, H, V} as their clients. The Border routers are E, A and C. There are no static routes and the flow is (B, E). After the BGP simulation, the forwarding graph is $B \to C \to V \to E$. Notice that the B packets leave the network at E and not at A which is B's selected next-hop. This is due to B's shortest path to A ($B \to C \to F \to W \to A$) which traverses C whose next-hop is E, rerouting B's origin forwarding plan to leave the network at E.

Initially, lines 2-5 gives us $\mathcal{H} = (E - V), (V - W), (W - A)$ to ensure the connectivity between the border routers and the route reflectors (represented in (iia) in Figure 3). Then the shortest path from B towards its selected next-hop is added (line 6) to ensure that the forwarding graph doesn't change due to increased IGP cost towards A. This adds $(B - C), (C - F), (F - W), (W - A)$ to the hot edges set. There is no static route, so the seventh line doesn't change $\mathcal{H}$. In line 8, we have that C (and E) is added to D_{NH} as in the forwarding graph the (B, C) is used where B and C have different next-hops. The shortest path of C (and E) towards its next-hop is added (for IGP cost reliability) to the hot edges. Considering the added short-

est path gives us the addition of the hot edges represented in Figure 3 (iib) giving us, in addition to the previously in Figure 3 (iib) the final set $\mathcal{H} = (E - V), (V - W), (W - A), (V - C), (C - F), (B - C), (F - W)$.

the example is added a bit in the rush, some adjustments might be necessary

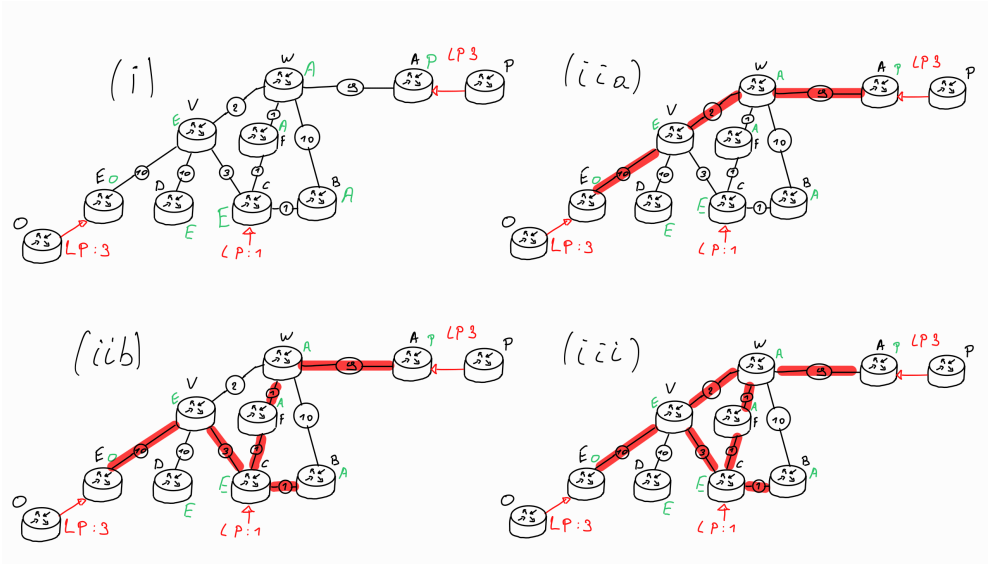


Figure 3: Exploration of the failure space with ϕ"Traffic from D to C traverses E". Crosses represents down links, padlocks represents up links. Absence of symbol represents undefined links. "Cold" in blue represents scenarios containing only cold edges (which stops the exploration).

Algorithm 2 Hot edges for BGP

procedure HOTBGP(u, d, E_{fwd}, l)
  $X \leftarrow$ nodes reachable from u under l
  $B_{Rl} \leftarrow \mathrm{TOP3}(B_R, X)$ ▷ subset of B_R under consideration of l
  $R_{Rl} \leftarrow R_R \cap X$ ▷ subset of R_R under consideration of l
  $\mathcal{H} \leftarrow \mathrm{AllSP}_l(R_{Rl}, B_{Rl}, l)$ ▷ ensure (i) connectivity
  $\mathcal{H} \leftarrow \mathcal{H} \cup \mathrm{SP}_l(u, \mathrm{NH}_d(u), l)$ ▷ initial shortest path to next-hop
  $\mathcal{S} \leftarrow \{y \mid (x, y) \in \mathit{Static}_d \ \cap \ E_{fwd}\}$ ▷ static route
  $\mathcal{D}_{\mathrm{NH}} \leftarrow \{y \mid (x, y) \in E_{fwd} \wedge \mathrm{NH}_d(x) \neq \mathrm{NH}_d(y)\}$ ▷ divergent next-hop
  for each $x \in \mathcal{D}_{\mathrm{NH}} \cup \mathcal{S}$ **do**
    $\mathcal{H} \leftarrow \mathcal{H} \ \cup \ \mathrm{SP}_l(x, \mathrm{NH}_d(x), l)$ ▷ ensure (ii) IGP costs unchanged
  if $R_{Rl} \ = \ \emptyset$ **then**
    $\mathcal{H} \leftarrow \mathcal{H} \cup \mathrm{AllSP}_l(\{u\}, B_{Rl}, l)$
  return $\mathcal{H}$

2.2 Complex Link Failure Probability model

2.3 Results

References

[1] Enke Chen, Tony J. Bates, and Ravi Chandra. *BGP Route Reflection: An Alternative to Full Mesh Internal BGP (IBGP)*. RFC 4456. Apr. 2006. DOI: 10.17487/RFC4456. URL: https://www.rfc-editor.org/info/rfc4456.

[2] Nick Feamster and Jennifer Rexford. "Network-Wide Prediction of BGP Routes". In: *IEEE/ACM Transactions on Networking* 15.2 (2007), pp. 253–266. DOI: 10.1109/TNET.2007.892876.

[3] Samuel Steffen et al. "Probabilistic Verification of Network Configurations". In: *Proceedings of the Annual Conference of the ACM Special Interest Group on Data Communication on the Applications, Technologies, Architectures, and Protocols for Computer Communication*. SIGCOMM '20. Virtual Event, USA: Association for Computing Machinery, 2020, pp. 750–764. ISBN: 9781450379557. DOI: 10.1145/3387514.3405900. URL: https://doi.org/10.1145/3387514.3405900.